

毕业论文（设计）任务书

学生姓名	曹志隆	学号	32301288	专业 班级	软件工程 2304	电话	13868360863
毕业论文（设计）中文题目		面向超级个体的 AIGC 协作开发工作流平台设计与实现					
毕业论文（设计）外文题目		Design and Implementation of an AIGC-Driven Collaborative Development Workflow Platform for Solo Developers					
指导教师 1	侯宏仑	职称	副教授	所在单位	计算机与计算科学学院	电话	13071858629
指导教师 2		职称		所在单位		电话	
选题类型	☒ 工程设计 ☐ 理论研究 ☐ 实验研究 ☐ 产品开发 ☐ 设计创作 ☐ 专题研究 ☐ 应用研究 ☐ 其他						
选题来源	☐ 国家重点研发计划项目 ☐ 国家社科规划、基金项目 ☐ 国家自然科学基金项目 ☐ 中央、国家各部门项目 ☐ 教育部人文、社会科学研究项目 ☐ 省（自治区、直辖市）项目 ☐ 国际合作研究项目 ☐ 港、澳、台合作研究项目 ☐ 企、事业单位委托项目 ☐ 外资项目 ☐ 国防项目 ☒ 学校自选项目 ☐ 非立项 ☐ 其他						
是否在实验、实习、工程实践和社会调查等社会实践中完成						☒ 是 ☐ 否	
毕业论文（设计）时间		【待填：学院规定起止日期】					
阶段工作内容	工作进度安排周次						
查阅文献	第 5 周（第七学期）至第 7 周（第七学期）						
文献综述和外文翻译	第 8 周（第七学期）至第 12 周（第七学期）						
开题报告	第 13 周（第七学期）至第 17 周（第七学期）						
研究、设计、开发	第 1 周（第八学期）至第 7 周（第八学期）						
撰写毕业论文（说明）	第 8 周（第八学期）至第 12 周（第八学期）						
答辩与修改	第 12 周（第八学期）至第 15 周（第八学期）						
“阶段工作内容”包括：查阅文献、调研、撰写综述、翻译外文资料、开题、实验或设计、绘图、撰写毕业论文或设计说明书、定稿及答辩等；							
论文（设计）的主要内容及要求：（以设计作品为主要成果形式的毕业设计要求以各分院实施方案为准）							
1、开题报告和文献阅读							
(1) 文献阅读：查阅文献 15 篇（含）以上，其中外文文献 2 篇（含）以上，近三年公开发表的文献 5 篇（含）以上，书籍不超过 3 本，期刊〔J〕和论文集〔C〕10 篇（含）以上，包括导师指定的全部参考文献。							
(2) 外文翻译：3000 字以上。							

(3) 文献综述：3000 字以上，包括国内外研究现状、研究方向、进展情况、存在问题和参考依据等。

(4) 开题报告：3000 字以上，包括选题的意义、可行性分析、研究内容、研究方法、拟解决的关键问题、预期结果和研究进度计划等。

2、论文：10000 字以上，包括绪论、正文、结论、参考文献等。法学、人文、经管类专业 8000 字以上，特殊专业应满足分院根据培养目标另行制定的相应要求。

3、课题要解决的主要问题和具体要求（填写如下）：

一、课题目标与工程评价维度

本课题面向独立开发者等“超级个体”的软件工程场景，拟设计并实现一个人工智能生成内容（Artificial Intelligence Generated Content, AIGC）协作开发 workflow 平台。平台不把大语言模型作为孤立的聊天工具，而是将其嵌入需求澄清、工作分解、上下文准备、代码实现、人工审查、测试和交付过程；通过模型上下文协议（Model Context Protocol, MCP）等方式集成代码智能体、Git/GitHub 及其他开源工具，形成可编辑、可执行、可介入、可追溯的开发 workflow。课题以可运行平台原型为主要成果，并从范围、开发效率、开发成本和开发质量四个维度验证该 workflow 相对于分散使用 AIGC 工具的改进效果。

前期工作仅用于验证项目建模、智能体工具调用、本地部署和代码工作区隔离等关键技术的可行性，并作为本课题方案选型与可行性分析的依据。毕业设计阶段将以任务书目标为准，重新完成范围收敛、核心交互原型、架构设计、系统实现和对照评价，最终成果和结论以本阶段形成的平台原型、测试及实验数据为准。

二、主要需求与原型交付模块

(1) 需求与用户体验（User Experience, UE）核心原型。围绕“创建或导入软件项目—澄清项目目标—生成并确认开发 workflow—执行任务—审查变更—测试与交付”的主线，形成用户角色、功能范围、异常场景和验收标准；完成项目入口、workflow、任务工作区、AI 协作活动流、差异审查和交付结果等核心界面原型，保证用户能够理解并控制 AIGC 的工作过程。

(2) AIGC 工作流与上下文模块。把项目目标组织为可独立验收的 Workstream 和 Task，由 AIGC 生成候选 workflow 并允许用户修改、拒绝或重新生成；由系统维护项目资料、决策、资产和运行记录，根据任务目标与权限构建结构化上下文包（Context Pack），避免完全依赖单次模型会话或无差别注入全部文件。

(3) 开源工具集成与工程执行模块。以可替换的代码智能体执行器为 AIGC 核心，通过 MCP、REST 及本地运行接口接入工具能力；在受管代码工作区中完成文件修改、命令运行、Git 差异、测试和 Draft Pull Request 交付，并保留后续接入其他模型、代码智能体和开源工具的适配边界。

(4) 质量治理与度量模块。对 AIGC 提议、工具调用、代码变更和外部交付设置人工确认、权限校验、写锁、checkpoint、Diff Review、撤销与恢复；记录任务耗时、模型调用与 Token、人工介入次数、测试结果、缺陷与返工、交付状态等证据，为范围、效率、成本和质量评价提供统一数据来源。

三、AIGC 驱动的技术路线

本课题采用“项目事实输入—AIGC 规划—受控执行—人工审查—测试交付—度量反馈”的技术路线：首先输入项目目标、已有代码、约束和验收标准，利用 AIGC 完成需求追问并生成 Project Brief；其次由 AIGC 生成可编辑的 Workstream/Task 工作流，用户确认后，系统按照任务范围组装 Context Pack 并配置允许使用的 MCP 工具；代码智能体在隔离工作区内执行任务，系统持续记录命令、文件、工具调用和阶段结果；用户通过审批与 Diff Review 介入关键决策，自动化测试通过后再形成 Git 提交或 Draft PR；最后将 Trace、交付物和度量数据沉淀为项目资产，用于复盘和下一轮 workflow 优化。

系统采用模块化前后端架构：前端使用 React、TypeScript 和 Vite 实现核心 UE，后端使用 Node.js 承载项目、workflow、上下文、AI 协作、权限和交付服务；AIGC 层通过 Runner Adapter、Context Pack 和 MCP 实现模型与工具解耦；执行层使用受管 checkout/worktree 及 Docker 隔离代码运行环境；数据默认本地持久化，并保留替换数据库和其他模型运行时的接口。

四、各开发环节拟解决的关键问题

- （1）需求与 UE 阶段：如何把用户的自然语言目标转化为范围明确、可编辑且可验收的开发 workflow，并通过核心原型让用户看清 AIGC 当前依据、执行状态和需要人工决策的位置。
- （2）架构设计阶段：如何让平台而非单一模型会话持有项目上下文和业务状态，保持代码智能体可替换；如何使页面、REST、MCP 和终端入口复用同一权限、状态与审计规则。
- （3）实现与集成阶段：如何在开放工具调用与本地安全之间取得平衡，控制文件、凭据、仓库和项目访问边界；如何处理并发写入、长任务中断、失败恢复、重复请求和代码交付一致性。
- （4）测试与评价阶段：如何构造相同任务下的对照基线，并采集足以解释范围完成度、效率变化、成本消耗和质量差异的证据，避免仅用功能数量、代码量或主观展示评价平台。

五、开发过程与进度要求

- （1）需求与核心原型：完成典型用户旅程、范围清单、非功能需求、界面信息架构和可交互 UE 原型，以核心链路可理解、可操作作为评审标准。
- （2）架构与最小闭环：完成领域模型、模块边界、接口和安全策略，优先实现“项目—workflow—任务—AIGC 执行—审查—测试—交付”的最小闭环。
- （3）迭代实现与开源集成：分阶段接入代码智能体、MCP 和 Git/GitHub 能力，每个增量均建立验收条件、自动化测试和失败恢复路径，避免在核心链路之外堆叠未经验证的功能。
- （4）实验、优化与交付：选择具有代表性的同类开发任务，在直接使用 AIGC 工具的基线流程与平台 workflow 之间进行对照；根据实验结果优化交互、上下文和执行策略，完成部署包、文档、测试报告和答辩材料。

六、预期成果与验收要求

- （1）提交可运行的 AIGC 协作开发 workflow 平台原型及源代码，能够演示项目接入、AIGC workflow 生成与确认、Context Pack、受控代码执行、人工审查、自动化测试和 Git 交付的完整链路。
- （2）提交需求规格、UE 核心原型、系统架构与数据模型、接口说明、部署运行手册、测试用例和实验报告；平台关键状态和论文结论应能追溯到代码、测试或运行证据。
- （3）建立四类评价指标：范围维度记录需求覆盖率、核心旅程完成率和范围变更情况；效率维度记录任务总耗时、迭代次数和人工介入次数；成本维度记录模型调用、Token、计算资源及部署成本；质量维度记录测试通过率、缺陷与返工、审查问题和交付完整性。
- （4）至少选取一组具有相近难度的开发任务，对比“分散使用 AIGC 工具”和“使用本平台 AIGC workflow”两种方式，说明实验环境、任务输入、评价口径、原始数据、结果与局限，不将自动化测试结果等同于真实用户效果。
- （5）系统应通过项目约定的静态检查、单元、集成和端到端测试，并验证权限、路径、敏感信息、审批、回滚和异常恢复；未执行的真实外部服务测试应明确标注，不得写成已通过。
- （6）完成不少于 10000 字的毕业论文（设计说明书），包括绪论、相关工作、需求与 UE 设计、系统架构、实现、测试与工程评价、结论与展望，并按 GB/T 7714—2015 著录参考文献。

推荐参考文献：

[1] WANG L, MA C, FENG X, et al. A survey on large language model based autonomous agents[J]. Frontiers of Computer Science, 2024, 18(6): 186345. DOI:10.1007/s11704-024-40231-1.

[2] XI Z, CHEN W, GUO X, et al. The rise and potential of large language model based agents: A survey[J]. Science China Information Sciences, 2025, 68(2): 121101. DOI:10.1007/s11432-024-4222-0.

[3] WANG Y, ZHONG W, HUANG Y, et al. Agents in software engineering: Survey, landscape, and vision[J]. Automated Software Engineering, 2025, 32(2): 70. DOI:10.1007/s10515-025-00544-2.

[4] QIAN C, LIU W, LIU H, et al. ChatDev: Communicative agents for software development[C]//Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics. Bangkok: Association for Computational Linguistics, 2024: 15174-15186. DOI:10.18653/v1/2024.acl-long.810.

[5] YANG J, JIMENEZ C E, WETTIG A, et al. SWE-agent: Agent-computer interfaces enable automated software engineering[C]//Advances in Neural Information Processing Systems 37. [S.l.]: Neural Information Processing Systems Foundation, 2024: 50528-50652. DOI:10.52202/079017-1601.

[6] BARKE S, JAMES M B, POLIKARPOVA N. Grounded Copilot: How programmers interact with code-generating models[J]. Proceedings of the ACM on Programming Languages, 2023, 7(OOPSLA1): 85-111. DOI:10.1145/3586030.

[7] VAITHILINGAM P, ZHANG T, GLASSMAN E L. Expectation vs. experience: Evaluating the usability of code generation tools powered by large language models[C]//CHI Conference on Human Factors in Computing Systems Extended Abstracts. New York: ACM, 2022: 1-7. DOI:10.1145/3491101.3519665.

[8] DODERLEIN J B, KOUADIO N H, ACHER M, et al. Piloting Copilot, Codex, and StarCoder2: Hot temperature, cold prompts, or black magic?[J]. Journal of Systems and Software, 2025, 230: 112562. DOI:10.1016/j.jss.2025.112562.

[9] CUI K Z, DEMIRER M, JAFFE S, et al. The productivity effects of generative AI: Evidence from a field experiment with GitHub Copilot[J/OL]. An MIT Exploration of Generative AI, 2024[2026-07-22]. DOI:10.21428/e4baedd9.3ad85f1c.

[10] AMERSHI S, WELD D, VORVOREANU M, et al. Guidelines for human-AI interaction[C]//Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems. New York: ACM, 2019: 1-13. DOI:10.1145/3290605.3300233.

[11] KLEPPMANN M, WIGGINS A, VAN HARDENBERG P, et al. Local-first software: You own your data, in spite of the cloud[C]//Proceedings of the 2019 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software. New York: ACM, 2019: 154-178. DOI:10.1145/3359591.3359737.

[12] VAN DER AALST W M P, TER HOFSTEDE A H M, KIEPUSZEWSKI B, et al. Workflow patterns[J]. Distributed and Parallel Databases, 2003, 14(1): 5-51. DOI:10.1023/A:1022883727209.

[13] MOREAU L, CLIFFORD B, FREIRE J, et al. The Open Provenance Model core specification (v1.1)[J]. Future Generation Computer Systems, 2011, 27(6): 743-756. DOI:10.1016/j.future.2010.07.005.

[14] SCULLEY D, HOLT G, GOLOVANOV E, et al. Hidden technical debt in machine learning systems[C]//Advances in Neural Information Processing Systems 28. Red Hook, NY: Curran Associates, 2015: 2503-2511.

[15] MODEL CONTEXT PROTOCOL. Model Context Protocol specification: 2025-06-18[EB/OL]. (2025-06-18)[2026-07-22]. <https://modelcontextprotocol.io/specification/2025-06-18>.

[16] WORLD WIDE WEB CONSORTIUM. PROV-DM: The PROV data model[S/OL]. (2013-04-30)[2026-07-22]. <https://www.w3.org/TR/2013/REC-prov-dm-20130430/>.

指导教师签名：_____

年 月 日

毕业论文（设计）工作指导小组意见：

☐ 同意下达任务书 ☐ 不同意下达任务书

负责人签名：_____

年 月 日